



Core Fundamentals

Before we proceed let's go back in the history and understand following things which we needed for Logistic Regression

- Log
- Growth
- Euler's Number i.e. e
- $\ln$ i.e. natural log
- Probability
- Odds
- Log of Odds
- Sigmoid Function
- ReLU Function
- tanh Function

Why we use Log?

$$10^1 = 10 \rightarrow \log_{10} 10 = 1$$

$$10^2 = 100 \rightarrow \log_{10} 100 = 2$$

$$10^3 = 1000 \rightarrow \log_{10} 1000 = 3$$

$$10^4 = 10000 \rightarrow \log_{10} 10000 = 4$$

$$10^5 = 100000 \rightarrow \log_{10} 100000 = 5$$

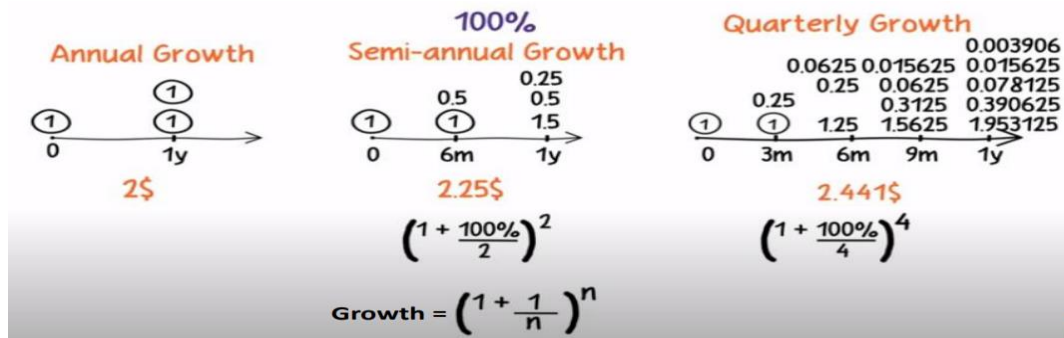
$$10^6 = 1000000 \rightarrow \log_{10} 1000000 = 6$$

To show a wide range of numbers in a small scale log is useful.



What is Growth?

Growth is always continuous in nature it will never be discrete.



What is Euler's Number?

Frequency	Time Period	Growth of 1\$
Yearly	1	2
Semi-Annually	2	2.25
Quarterly	4	2.44140625
Monthly	12	2.61303529
Weekly	52	2.692596954
Daily	365	2.714567482
Hourly	8760	2.718126692
Minute Wise	525600	2.718279243
Second Wise	31536000	2.718281781

Max Growth Possible is 171.82%
 $e = 2.718281781$

e → Maximum, Continuous, 100%, One time compounding growth period

What is Natural Log?

$$e^1 = 2.718 \rightarrow \ln_e 2.718 = 1$$

$$e^2 = 7.38905 \rightarrow \ln_e 7.38905 = 2$$

$$e^3 = 20.0855 \rightarrow \ln_e 20.0855 = 3$$

$$e^4 = 54.5981 \rightarrow \ln_e 54.5981 = 4$$

$$e^5 = 148.4130 \rightarrow \ln_e 148.4130 = 5$$



Natural log is used to show continuous growth for a wide range of number in a smaller scale.

What is probability?

Probability measures how likely an event is to happen?

0 → the event will *never* happen

1 → the event will *definitely* happen

Anything in between → shows how likely the event will occur

Simple Example:

If you toss a fair coin:

- Probability of getting **heads** = $1/2 = 0.5$
- Probability of getting **tails** = $1/2 = 0.5$

Basic Formula

$$\text{Probability} = \frac{\text{Number of favorable outcomes}}{\text{Total number of possible outcomes}}$$

Another Example

Rolling a fair 6-sided die:

- Probability of getting a **3** = $1/6$
- Probability of getting an **even number (2, 4, 6)** = $3/6 = 1/2$

Where Probability is used

- Weather forecasts (chance of rain ☁️)
- Games and gambling 🎲
- Risk analysis (insurance, finance)
- Science and data analysis

What are odds?



- Odds are another way of expressing probability, but instead of comparing success to all outcomes, they compare:

Success vs Failure

Formula

$$\text{Odds} = \frac{p}{1 - p}$$

where p = probability of the event.

Example:

If probability of rain is 0.75:

- Probability (p) = 0.75
- Failure (no rain) = 0.25

$$\text{Odds} = \frac{0.75}{0.25} = 3$$

So the odds are **3 to 1 (3:1)** → rain is 3 times more likely than no rain.

Another Example:

If probability = 0.5

$$\text{Odds} = \frac{0.5}{0.5} = 1$$

Odds = **1:1** (equal chance)

What is Log of odds?



The **log of odds** (also called **logit**) is just taking the logarithm of the odds.

$$\text{Log-odds} = \log \left(\frac{p}{1-p} \right)$$

Why take the log?

Because:

- Probability is limited between **0 and 1**
- Odds go from **0 to ∞**
- Log-odds go from **$-\infty$ to $+\infty$**

This makes mathematical modelling much easier.

Illustration:

Probability (P)	1 - Probability	Probability (P) / (1 - Probability (P))	log (p/1-p)
0	1	0	$-\infty$
0.1	0.9	0.111111111	-2.197224577
0.2	0.8	0.25	-1.386294361
0.3	0.7	0.428571429	-0.84729786
0.4	0.6	0.666666667	-0.405465108
0.5	0.5	1	0
0.6	0.4	1.5	0.405465108
0.7	0.3	2.333333333	0.84729786
0.8	0.2	4	1.386294361
0.9	0.1	9	2.197224577
1	0	∞	∞

What is Sigmoid Function?

The **sigmoid function** is a special mathematical function widely used in Machine Learning and Statistics to convert any real number into a value between **0 and 1**. But why? We will understand this with practical case studies later...

Sigmoid Function:

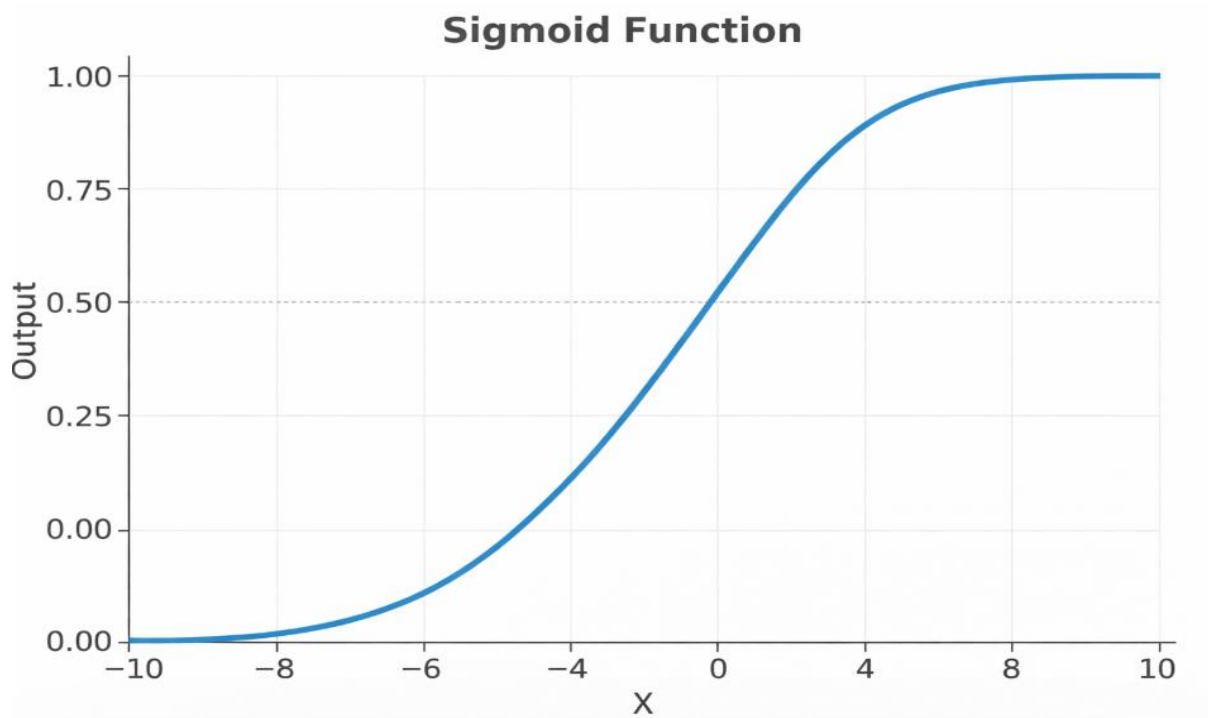
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Where:

x can be any real number

e is a mathematical constant (~ 2.718)



What is does?

It takes any input:

- very large negative $\rightarrow$ output ≈ 0
- $0 \rightarrow$ output = **0.5**
- very large positive $\rightarrow$ output ≈ 1

So it "squashes" values into the range $(0, 1)$.

Understand with Example:



x	e	e ^{-x}	1+e ^{-x}	1/(1+e ^{-x})
-200	2.718	7.22597E+86	7.22597E+86	1.3839E-87
-100	2.718	2.68812E+43	2.68812E+43	3.72008E-44
-50	2.718	5.18471E+21	5.18471E+21	1.92875E-22
-10	2.718	22026.46579	22027.46579	4.53979E-05
-9	2.718	8103.083928	8104.083928	0.000123395
-8	2.718	2980.957987	2981.957987	0.00033535
-7	2.718	1096.633158	1097.633158	0.000911051
-6	2.718	403.4287935	404.4287935	0.002472623
-5	2.718	148.4131591	149.4131591	0.006692851
-4	2.718	54.59815003	55.59815003	0.01798621
-3	2.718	20.08553692	21.08553692	0.047425873
-2	2.718	7.389056099	8.389056099	0.119202922
-1	2.718	2.718281828	3.718281828	0.268941421
0	2.718	1	2	0.5
1	2.718	0.367879441	1.367879441	0.731058579
2	2.718	0.135335283	1.135335283	0.880797078
3	2.718	0.049787068	1.049787068	0.952574127
4	2.718	0.018315639	1.018315639	0.98201379
5	2.718	0.006737947	1.006737947	0.993307149
6	2.718	0.002478752	1.002478752	0.997527377
7	2.718	0.000911882	1.000911882	0.999088949
8	2.718	0.000335463	1.000335463	0.99966465
9	2.718	0.00012341	1.00012341	0.999876605
10	2.718	4.53999E-05	1.0000454	0.999954602
50	2.718	1.92875E-22	1	1
100	2.718	3.72008E-44	1	1
200	2.718	1.3839E-87	1	1

What is ReLU function?

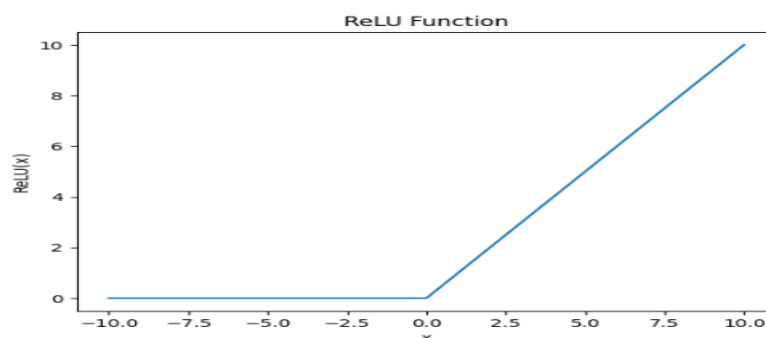
The **ReLU function** (Rectified Linear Unit) is one of the most commonly used activation functions in Machine Learning, especially in neural networks. It simply **removes all negative values** and keeps positive ones unchanged.

$$\text{ReLU}(x) = \max(0, x)$$

If $x > 0 \rightarrow \text{output} = x$

If $x \leq 0 \rightarrow \text{output} = 0$

The graph output looks like given below....





What is tanh i.e. hyperbolic tangent function?

The **tanh function** (hyperbolic tangent) is another important activation function used in Machine Learning and neural networks. It gives the value between -1 to +1

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Large negative input → output ≈ -1

Input = 0 → output = 0

Large positive input → output ≈ +1

So it **squashes values between -1 and 1**

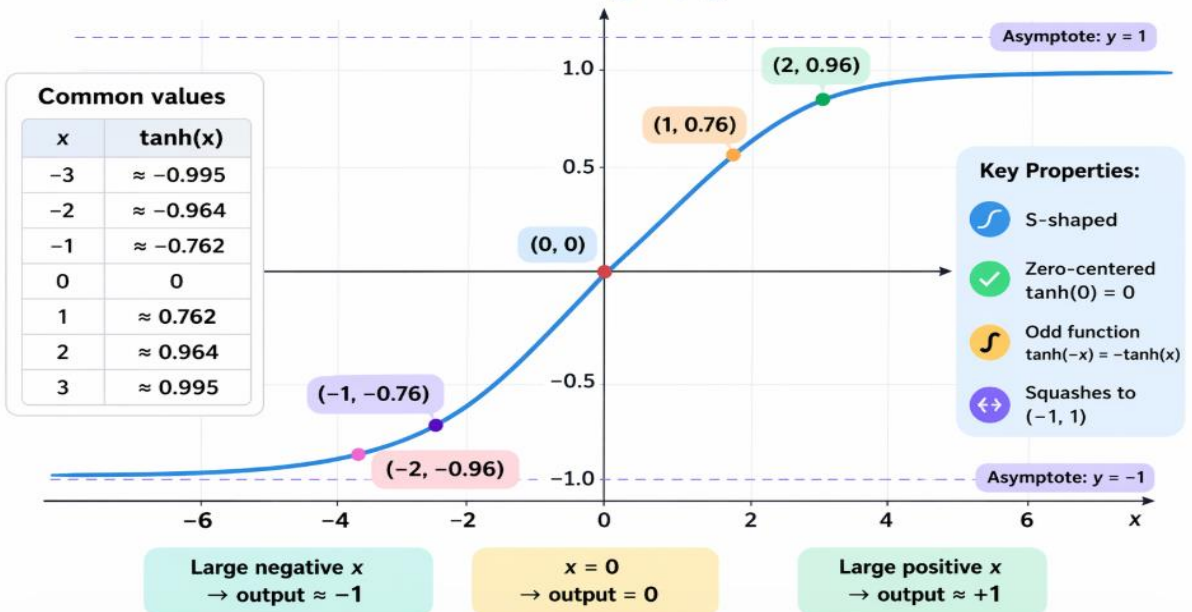
Understand with example:

X	e^x	e^{-x}	$e^x - e^{-x}$	$e^x + e^{-x}$	$e^x - e^{-x} / e^x + e^{-x}$
-200	1.3839E-87	7.22597E+86	-7.22597E+86	7.22597E+86	-1
-100	3.7201E-44	2.68812E+43	-2.68812E+43	2.68812E+43	-1
-50	1.9287E-22	5.18471E+21	-5.18471E+21	5.18471E+21	-1
-10	4.54E-05	22026.46579	-22026.46575	22026.46584	-0.9999999996
-9	0.00012341	8103.083928	-8103.083804	8103.084051	-0.999999997
-8	0.00033546	2980.957987	-2980.957652	2980.958323	-0.9999999775
-7	0.00091188	1096.633158	-1096.632247	1096.63407	-0.999998337
-6	0.00247875	403.4287935	-403.4263147	403.4312722	-0.999987712
-5	0.00673795	148.4131591	-148.4064212	148.419897	-0.999909204
-4	0.01831564	54.59815003	-54.57983439	54.61646567	-0.9993293
-3	0.04978707	20.08553692	-20.03574985	20.13532399	-0.995054754
-2	0.13533528	7.389056099	-7.253720816	7.524391382	-0.96402758
-1	0.36787944	2.718281828	-2.350402387	3.08616127	-0.761594156
0	1	1	0	2	0
1	2.71828183	0.367879441	2.350402387	3.08616127	0.761594156
2	7.3890561	0.135335283	7.253720816	7.524391382	0.96402758
3	20.0855369	0.049787068	20.03574985	20.13532399	0.995054754
4	54.59815	0.018315639	54.57983439	54.61646567	0.9993293
5	148.413159	0.006737947	148.4064212	148.419897	0.999909204
6	403.428793	0.002478752	403.4263147	403.4312722	0.999987712
7	1096.63316	0.000911882	1096.632247	1096.63407	0.999998337
8	2980.95799	0.000335463	2980.957652	2980.958323	0.999999775
9	8103.08393	0.00012341	8103.083804	8103.084051	0.999999997
10	22026.4658	4.53999E-05	22026.46575	22026.46584	0.9999999996
50	5.1847E+21	1.92875E-22	5.18471E+21	5.18471E+21	1
100	2.6881E+43	3.72008E-44	2.68812E+43	2.68812E+43	1
200	7.226E+86	1.3839E-87	7.22597E+86	7.22597E+86	1



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Range: $(-1, 1)$



What is softmax Function?

Given a vector of values $z_1, z_2, \dots, z_n$, the softmax function converts them into probabilities like this:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Intuition

- It takes **raw scores (logits)** and turns them into **probabilities**.
- Each output is:
 - Between **0 and 1**
 - All outputs **sum to 1**

Example

Suppose a model outputs:

- Class A: 2.0
- Class B: 1.0
- Class C: 0.1

After applying softmax, you might get:

- Class A: 0.66
- Class B: 0.24
- Class C: 0.10

Now these values can be interpreted as **probabilities**.



Understand with Example:

A neural network sees an image and produces these raw scores (called *logits*):

Animal Score

Cat 2.0

Dog 1.0

Bird 0.1

These are not probabilities yet.
Softmax converts them into probabilities.

Step 1: Apply Exponential e^x

Softmax first computes e^{score} .

$$e^{2.0} \approx 7.39$$

$$e^{1.0} \approx 2.72$$

$$e^{0.1} \approx 1.105$$

So now we have:

Animal	e^{score}
Cat	7.39
Dog	2.72
Bird	1.105



Step 2: Add Them

$$7.39 + 2.72 + 1.105 = 11.215$$

Step 3: Divide Each by the Total

For Cat:

$$\frac{7.39}{11.215} \approx 0.659$$

For Dog:

$$\frac{2.72}{11.215} \approx 0.242$$

For Bird:

$$\frac{1.105}{11.215} \approx 0.099$$

✓ Final Softmax Output

Animal	Probability
Cat	65.9%
Dog	24.2%
Bird	9.9%

Now the model says:

"I think this image is a cat with 65.9% confidence."

✚ Why Exponential?

Exponentials make larger numbers grow much faster.

Example:

Score	e^x
1	2.7
2	7.4
5	148

So the highest score becomes much more dominant.

*** The End***